Infrastructure Invention Pattern

A design pattern for infrastructure-grade inventions

基礎設施級發明的設計模式

`Problem → Property → Analogy → Protocol → Invention`

v1.0 · June 2026

Developed during the OpenRNG Computational Trust project

Applicable to any infrastructure domain

The Core Insight

> **Features are implementations — they can be designed around.**
> **Properties are characteristics — they define what a system *is*.**
> **Properties survive technology changes. Features don't.**

Most inventions are described as features: "a system that does X." Infrastructure-grade inventions are described as properties: "a system that *has* X." Docker's feature is containers. Its property is portability. Git's feature is commits. Its property is immutable history. TLS's feature is the handshake. Its property is confidentiality.

This pattern provides a repeatable methodology for designing inventions at the property level — ensuring that each invention solves a real technical problem, creates a named system property, can be independently remembered through analogy, and can be implemented and extended by anyone.

The Five Layers of an Invention

A mature invention exists simultaneously at five layers. Each layer serves a different audience and purpose. Mixing layers weakens all of them.

Layer 1 — Technical Problem

AUDIENCE: PATENT EXAMINERS · ATTORNEYS · ENGINEERS

What it answers: What can't be done today?

Test: Can you say it in one sentence? Is it a problem, not a feature? Does it exist without your specific technology?

Example: "Random values can be previewed before commitment."

Layer 2 — System Property

AUDIENCE: PAPERS · STANDARDS · CONFERENCES · INDUSTRY

What it answers: What property does solving the problem create?

Test: Is it a property of the system, not a description of the implementation? Would a competitor need to achieve the same property to solve the same problem?

Example: "Dual Temporal Integrity"

Layer 3 — Analogy

AUDIENCE: EVERYONE — THE WAY THE WORLD REMEMBERS

What it answers: What familiar concept does this most resemble?

Test: Can someone who has never heard of your invention understand the core idea in 10 seconds through the analogy?

Example: "Like E2E Encryption, but for trust" · "Like Battery Health, but for trust infrastructure"

Industry doesn't remember properties. Industry remembers analogies. JWT is remembered as "passport for APIs," not as "portable signed identity claims."

Layer 4 — Protocol / RFC

AUDIENCE: IMPLEMENTERS · STANDARDS BODIES · ENGINEERING TEAMS

What it answers: How exactly does it work?

Test: Can a competent engineer implement it from this specification alone, without talking to the inventor?

Example: RFC-0001: VEO-1 Specification

Layer 5 — Patent

AUDIENCE: PATENT OFFICES · ATTORNEYS · LICENSEES

What it answers: What specific claims are protected?

Test: Do the claims naturally follow from the Problem + Property? Are they broad enough to cover alternative implementations but specific enough to survive examination?

Example: "A method for generating batch random numbers with dual temporal integrity guarantees..."

The Readiness Test

> **If you have only the Problem, you have a bug report.**
>
> **If you have only the Property, you have marketing.**
>
> **When you have both, you have an invention.**
>
> **If you can teach it, you have a standard.**
>
> Protocols, patents, and adoption naturally follow.

This test prevents two common mistakes:

- **Filing too early:** Having a Problem without a Property means you know what's broken but haven't yet defined what "fixed" looks like as a system characteristic. The patent will describe an implementation, not an invention.
- **Filing too late:** Having a Property without a filed Problem means competitors may define the same property first. The Problem locks your priority date; the Property guides your claim strategy.

The Pressure Test

Before committing resources to any invention, apply these five questions:

#	QUESTION	WHY IT MATTERS	RED FLAG
1	Does this problem exist today?	Not a hypothetical need	"In the future, someone might need..."
2	Does it exist without your specific technology?	Problem must be bigger than your solution	"This problem only matters for our product"
3	Is it a technical problem, not a business problem?	Patents require technical effect	"Customers want..." instead of "Systems cannot..."

#	QUESTION	WHY IT MATTERS	RED FLAG
4	Can it generate many implementations?	Broad claim potential	Only one way to solve it
5	Can you say it in one sentence?	Core is focused enough	Requires a paragraph to explain

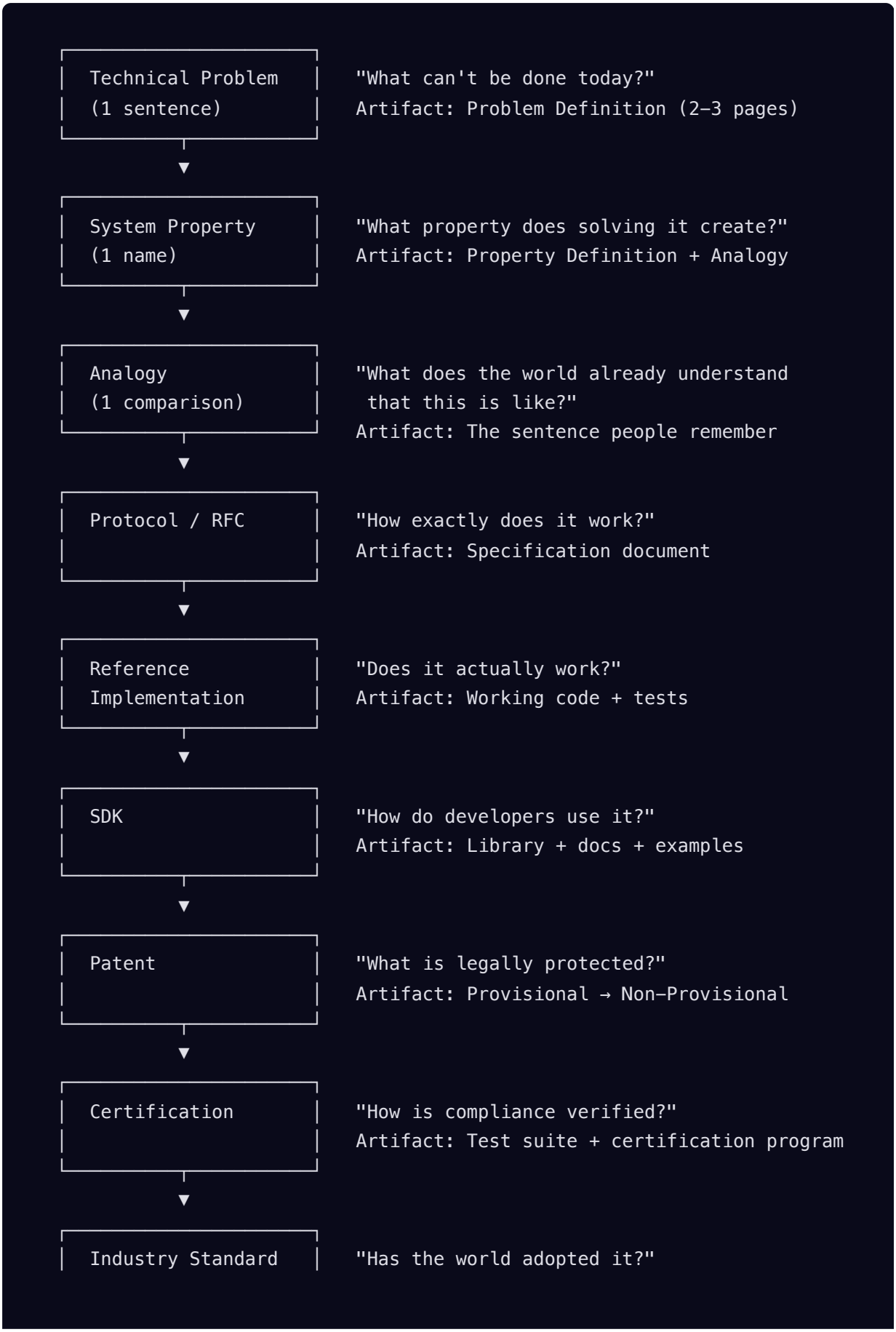
An invention that passes all five questions is ready for the Problem → Property → Protocol → Patent pipeline. An invention that fails any question needs more refinement before resources are committed.

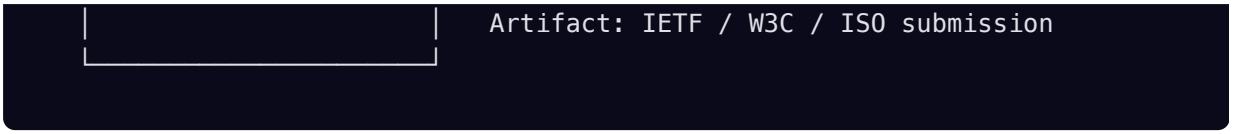
Where Each Layer Lives

DOCUMENT TYPE	PROBLEM	PROPERTY	ANALOGY	PROTOCOL	PATENT
Patent application	✅ Primary	Background	❌	Embodiment	✅ Claims
RFC / Specification	Motivation	✅ Title	Intro	✅ Primary	❌
Whitepaper	✅ Ch.1	✅ Primary	✅ Examples	Summary	❌
Conference talk	Hook	✅ Primary	✅ Primary	Demo	❌
Manifesto	Background	Beliefs	✅ Primary	❌	❌
Investor deck	Slide 2	Slide 3	✅ Slide 1	Appendix	Appendix

The Full Lifecycle

An infrastructure-grade invention follows a lifecycle from first insight to industry standard. Each stage produces a concrete artifact. The stages are sequential — skipping stages produces fragile inventions.


Technical Problem
(1 sentence)
"What can't be done today?"
Artifact: Problem Definition (2–3 pages)
System Property
(1 name)
"What property does solving it create?"
Artifact: Property Definition + Analogy
Analogy
(1 comparison)
"What does the world already understand
that this is like?"
Artifact: The sentence people remember
Protocol / RFC
"How exactly does it work?"
Artifact: Specification document
Reference
Implementation
"Does it actually work?"
Artifact: Working code + tests
SDK
"How do developers use it?"
Artifact: Library + docs + examples
Patent
"What is legally protected?"
Artifact: Provisional → Non-Provisional
Certification
"How is compliance verified?"
Artifact: Test suite + certification program
Industry Standard
"Has the world adopted it?"



Not every invention reaches Industry Standard. But every invention that does reach it followed this sequence — or wishes it had. Note: Reference Implementation and SDK come before Patent — this matches how infrastructure standards actually form (Linux, Kubernetes, OpenTelemetry). Implementation proves the invention works; the patent protects it.

Applied Example: OpenRNG Patent Family

#	TECHNICAL PROBLEM	SYSTEM PROPERTY	ANALOGY
1	Random values cannot be individually verified	**Verifiable Randomness**	Like HTTPS verified connection
2	Random values can be previewed before commitment	**Dual Temporal Integrity**	Like a tamper-evident seal (before + after)
3	Trust disappears after randomness is consumed	**Portable Trust Assertion**	Like JWT for identity → VEO for uncertainty
4	Trust cannot survive delegation	**End-to-End Verifiable Trust**	Like E2E Encryption for trust chains
5	Trust quality silently degrades	**Continuous Trust Health**	Like Battery Health for trust infrastructure

Five problems. Five properties. Five analogies. All derive from one architecture. None overlaps with another.

Failure Modes

Each layer has a characteristic failure. Recognizing these failures early prevents wasted resources.

LAYER	FAILURE MODE	SYMPTOM	FIX
Problem	"This only matters for my product"	Problem disappears if your company disappears	Restate the problem without mentioning your solution. If you can't, it's not a real problem — it's a product requirement.
Property	"This is just another feature"	Property can be described as a verb ("it does X") instead of an adjective ("it is X")	Ask: "Would a competitor solving the same problem need to achieve the same property?" If yes, it's a property. If no, it's an implementation detail.
Analogy	"The analogy requires explanation"	You say "it's like X" and then spend 5 minutes explaining why	The analogy should produce a nod in 10 seconds. If it doesn't, either the analogy is wrong or the invention isn't clear enough yet.
Protocol	"Only the inventor can implement it"	The specification has implicit knowledge that isn't written down	Hand the spec to an engineer who wasn't involved. If they can't implement it, the spec is incomplete.
Patent	"Claims only cover one implementation"	A competitor can solve the same problem, achieve the same property, and not infringe	Claims should protect the Property, not the Protocol. The Protocol is one way to achieve the Property; claims should cover all reasonable ways.

The meta-failure: Mixing layers. Using Movement language in patent claims. Using Patent language in conference talks. Using Analogy language in specifications. Each layer has its own audience and its own vocabulary. Respect the boundaries.

Infrastructure Invention Pattern v1.0

Developed during the OpenRNG Computational Trust project · June 2026

Applicable to any infrastructure domain

Problem → Property → Analogy → Protocol → Invention